

Generation of Animations Between Images Using a Genetic Algorithm

Jesse Perkins

Undergraduate Student

Computing Science, Thompson Rivers University

Timothy Britcliffe

Undergraduate Student

Computing Science, Thompson Rivers University

Justin Stehling

Undergraduate Student

Computing Science, Thompson Rivers University

April 21 2026

1 Problem Description

The generation of fluid, frame-by-frame sequences is typically a very time consuming and tedious task, requiring significant precision from skilled artists. Conventional techniques currently consist of either creating each frame by hand, or trusting a computer that struggles to maintain a realistic sequence. Other issues regarding using tools to help generate images involve computational cost, most end up being extremely computationally expensive.

We focused our research on working on an evolutionary approach to animation generation. The goal is that you can submit any two photos to our program with a selected size, and as long as the two images have a clear path between the two (and aren't just two mutually exclusive photos), the genetic algorithm will be able to produce a visually sound animation.

The framework we have built lowers computational cost by processing images in chunks in parallel which takes significantly less time in comparison to processing a whole image at a time. It also aims to guarantee convergence via our fitness metrics, so we are achieving at least a somewhat comprehensible animation; still heavily dependent on the input images.

2 Method

This section discusses the implementation we used to build a solution to our problem description

2.1 Genetic Algorithm

The individuals are represented as 32-bit binary strings, the initial 16 bits encode the radius (r) of the hypersphere and the remaining 16 bits encode the increment (Δ). Each segment is converted into a decimal integer and normalized using constants 10^5 and 10^6 respectively which follows the methodology described in [1].

The algorithm uses a fixed population size of 30 individuals. To enhance the computational efficiency and stability, we utilize a spatial decomposition strategy where we divide the input data into subdomains defined by *subsection_size*. This methodology allows us to mitigate the risk of divergence that typically follows computing large-scale Jacobian inversion, and allows for parallelized processing on CPU-bound devices.

We calculate the λ values of each subdomain independently of one another. The goal of the algorithm is to achieve $\lambda = 1.0$, which identifies a successful transformation as 1.0 represents the IMG_2_arr. To define the fitness score of an individual we take the subdomain λ which is the smallest value and score it using the following equation:

$$Fitness = 1/(1 + |1 - \lambda_{min}|)$$

This makes certain that an individual is only ranked high if the least stable segment still converges reliably, which means the remaining subdomains also all converge reliably.

Selection is then performed via an elitist-based strategy. We select the 10 individuals with the highest fitness scores to produce 20 (a 2:1 child to parent ratio) offspring through crossover. We then use a mutation rate of 0.01 (or 1%) to give genetic diversity and variation. To preserve exploration we then select a stochastic sample of 10 individuals from the remaining population. This approach maintains the population size of 30, while promoting growth and evolution.

2.2 Previous Works

When finding sources to base our project on and expand upon, we found two previous papers/research that we decided would be the most useful in helping us achieve our goal.

The first paper we looked at by Hernandez-Mendez et al. focused on the animation of hand gestures in sign language, where a genetic algorithm was used in combination with a system of equations to create the animations from one letter to another [2].

The second paper by Torres-Muñoz et al. was a reference paper used in the paper highlighted above, that went into more detail about how to translate images into a system of equations that could also be used to create an animation between the two images [1].

2.3 Mathematical Foundation

The mathematical foundation has three major points: translating images into vectors, establishing a homotopy and hypersphere equation, and then solving the hypersphere equation using the Newton-Raphson method [1].

For the image translation process, we translate the key frame images into vectors where each pixel is translated into a number between $[0, 255]$ and would result in the general form vector of length n :

$$[x_1 x_2 x_3 \dots x_n]$$

To establish a homotopy equation, we first establish the general homotopy form to create a value (λ) that can transition from the starting image to the ending image, where if λ equals 0 it will result in the starting image, and if λ equals 1 it will result in the ending image:

$$H(x, \lambda) = \lambda(IMG_2) + (1 - \lambda)(IMG_1)$$

We then process each image by multiplying it by a random coefficient matrix **A** (which has a width and height of n , the size of the image vectors) and multiplying it by the images to get matrices **B** (from the starting image) and **D** (from the ending image) [1]. Additionally, since each pixel (or x-value) of the starting image corresponds to the same pixel in the ending image, we can say that they are the same variable [1]. In the end, we can say that the homotopy equation can be represented as:

$$H(x, \lambda) = \lambda(Ax - D) + (1 - \lambda)(Ax - B)$$

In order to track the homotopy curve between each image, the hypersphere equation is introduced to get intermediate pixel values that can be used in the making of the animation between key frames [1]. Below, x represents an individual pixel, C represents the center point of the hypersphere, and r is the radius of the hypersphere:

$$(x_1 - C_1) + (x_2 - C_2) + (x_3 - C_3) + \dots + (x_{n+1} - C_{n+1}) - r^2 = 0$$

After substituting $(x_{n+1} - C_{n+1})$ with λ^2 , we get the following system of equations:

$$H(x, \lambda) = \lambda(Ax - D) + (1 - \lambda)(Ax - B)$$

$$\sum_{i=1}^n (x_i - C_i) + \lambda^2 - r^2 = 0$$

And F is the full system of equations (n is the length of the image vectors and k is the total number of equations in the system):

$$\begin{aligned}
F_1 &= \lambda(Ax - D)_1 + (1 - \lambda)(Ax - B)_1 \\
F_2 &= \lambda(Ax - D)_2 + (1 - \lambda)(Ax - B)_2 \\
F_3 &= \lambda(Ax - D)_3 + (1 - \lambda)(Ax - B)_3 \\
&\dots \\
F_{k-1} &= \lambda(Ax - D)_n + (1 - \lambda)(Ax - B)_n \\
F_k &= (x_1 - C_1) + (x_2 - C_2) + (x_3 - C_3) + \dots + (x_{n+1} - C_{n+1}) - r^2
\end{aligned}$$

In order to process them to get the intermediate frames for the animation, we need to solve the hypersphere equation using the Newton-Raphson method, where the general form is:

$$x_{n+1} = x_n - \frac{F(x_n)}{F'(x_n)}$$

However, since the hypersphere equation is non-linear, the general form can't be applied in this situation. Instead, you need to use the Jacobian matrix to find the next x -value, so the general form becomes:

$$x^{(k+1)} = x^{(k)} - J(x^{(k)})^{-1}F(x^{(k)})$$

Additionally, after looking at the Jacobian matrix for simple examples (1x1 and 2x2 arrays), they followed a general pattern that made it so that we could skip the calculations of partial derivatives. The general pattern of the Jacobian matrix is as follows, where d is the difference between the resulting matrices $\mathbf{B}$ and $\mathbf{D}$, and n and m is the length of the image vector x :

$$\begin{aligned}
&[A_{11}A_{12}...A_{1n}d_1] \\
&[A_{21}A_{22}...A_{2n}d_2] \\
&[A_{31}A_{32}...A_{3n}d_3] \\
&\dots \\
&[A_{m1}A_{m2}...A_{mn}d_n] \\
&[(x - C)_1^2(x - C)_2^2...(x - C)_n^2\lambda^2]
\end{aligned}$$

While this gives us a way to calculate the next value, matrix division is very computationally intensive and will vastly increase the amount of time to calculate a full transition when carried out for 30 iterations. Therefore, if we rearrange the equation in the sequence of steps shown below, we can simplify this into a linear system of equations that make the calculations a lot more straightforward:

$$x^{(k+1)} - x^{(k)} = -J(x^{(k)})^{-1}F(x^{(k)})$$

$$\begin{aligned}\Delta x &= -J(x^{(k)})^{-1}F(x^{(k)}) \\ J(x^{(k)})\Delta x^{(k)} &= (-F(x^{(k)}))\end{aligned}$$

Now that it is in linear form, the calculations now become a lot easier and faster. Once solved, we will get the change in x and λ . All we need to do is add them to the current values of x and λ , and increment the radius to complete the iteration. Now we go through the same process with the new values for x , λ , and r for another 29 iterations, incrementing the radius at the start of each iteration for a total of 30 iterations.

3 Materials

For this project, we had a good amount of materials/resources we used/accessed to help us with our implementation. Personal consisted of Jesse, Timothy, Justin primarily, and we consulted with Richard Brewster, Professor, Department of Mathematics and Statistics, Thompson Rivers University, who provided insight, validation, and a general path of what we were doing with our Mathematical Foundation. On top of personnel we utilized the two papers we were building the core fundamentals of our project around, S. Hernandez-Mendez et al [1] and D. Torres-Muñoz et al [2].

On top of these personnel and informational materials/resources, we utilized personal laptops/computers, Thompson Rivers University computers, Git/GitHub, Trello (strictly for project organization), and our respective IDE's we chose (VS-Code, PyCharm, etc.).

All of these materials were vital to the creation and success we have had with our project.

4 Evaluation

We evaluated the fitness of the output in a rather different way. As discussed prior, we divided the images into subdomains (slices) for processing. This allowed us to run each subdomain in parallel for better computational efficiency, which also enabled better accuracy as processing as a whole can lead to deviations. We evaluated the resulting output (an animation between two input images) from a subjective visual perspective. It is hard to categorize in computer language how to view and determine if a .GIF or animation file actually makes coherent sense as it traverses the sequence of photos.

Due to these limitations, our approach involved us three viewing the animations and determining whether they are visually appealing and have a cohesive flow. Not all of the animations we were generating made sense, or fit our subjective "good" categorization, however we had a couple that did fit this categorization. We found that the inputs and variation in the input images drastically impacted the quality of the resulting animation. We discovered that colour shifts may have drastic impacts, as well as just general pieces randomly "moving". We used a variety of libraries to help us, we used NumPy for the

matrices and vectors as well as math computing, we used joblib to enable parallel processing, and we used Pillow for our image processing, image creation, and .GIF creation.

We built off of past research by introducing an optimized fitness function which used slices instead of processing an entire image at once. We also used a different selection algorithm for creating the next generation, the paper we used which had the genetic algorithm used an elitist approach. We get the best 10 individuals for breeding (2:1 children to parent), but we also select 10 random individuals from the remaining individuals in the prior population. These modifications allow us to achieve a better efficiency, while also maintaining a better genetic diversity/variation by ensuring that the generations to follow won't just be clones and replicated individuals with minute changes.

5 Limitations

There were several limitations faced in our works. The most major issue being the cementing of the mathematical concepts used to calculate the intermediate pixels between key frames. The research papers had helpful overarching ideas and theories about implementation. However, after closer inspection, we discovered that these papers had significant flaws in their examples or methods. For example, while the Newton-Raphson method was mentioned to solve the hypersphere equation, they did not provide examples of such calculations. This made the project much more difficult as we had to figure out ourselves that we needed to use a Jacobian matrix to solve each iteration.

Another limitation we faced was when we divided an individual into subdomains. Since each subdomain was calculated in isolation, this meant that each chunk followed their own homotopical path, and therefore some subdomains would converge at $\lambda = 1$ before others. This led to some parts of the generated animation to be quite odd and out of place with the rest of the animation, and with each chunk having its own separate lambda value at the end of an iteration it made scoring an individual much tougher as there are numerous ways to go about picking the lambda, or recompiling into one that goes on to be used to score the individual.

Lastly, the first few iterations repeatedly showed that changes in x and λ are quite large, and would result in colours of pure white or black before rapidly decreasing and slowly converging to the second image by the 30th iteration. This led to us skipping the first set of iteration values entirely which shortened the number of usable frames that could be used in the final animation.

6 Gallery

We were inspired by one of our teammates' desire to create something cool and another's desire to create an animation with his own art with minimal effort. Our genetic algorithm isn't trained on any art style, they build off of the

given art that we provide. It is flawed and therefore human, producing visual animation not unlike what you would see in a dream. While not perfect, this method could be used to animate things in a variety of modern applications from games to movies.

Beneath the animation and images is the blood, sweat, and incredible amount of tears that went into this project, not from the math from the research papers that was incorrect or poorly explained but the passion of its creators.

- Exhibit A: Annoyed girl
<https://media.giphy.com/media/BpUeoEuKxy982a2oTz/giphy.gif>
<https://media.giphy.com/media/S3CaLU4RuQqDN4nK9V/giphy.gif>
<https://media.giphy.com/media/mhelDtej3amo0CH3Pz/giphy.gif>
- Exhibit B: Eye Closing
<https://media.giphy.com/media/C92CFRrsQkMIDdpUAZ/giphy.gif>
- Exhibit C: Cannon Firing:
<https://media.giphy.com/media/3824N45dmOPif0dcNG/giphy.gif>

7 Next Steps

While we were successful in what our goals were, there are several changes that we could do in the future to enhance the project's capabilities. Some of these changes include:

- **Sequence of Preset lambda:** Discover a preset sequence of lambda values that would result in a natural animation and only calculate the x-values at each λ value in the preset sequence. This could decrease the computation needed, make the process more straightforward, and could improve the resulting animation
- **Subdomain Syncing:** Sync each subdomain (slice/chunk) so that it is more in-line with other chunks from the individual. This could be done by running an iteration calculation prematurely to discover the smallest λ value, and then apply that value across all other chunks, calculating the values at that specific λ value. This would mean each chunk is at the same point and would be synchronous with one another for every iteration, even if the calculation time is increased.
- **Score Creation:** There are a variety of ways to score an individual based on each slice's lambda, for this project we chose the minimum lambda and got excellent results. However, fine-tuning this further and experimenting with other ideas on scoring an individual from the list of subdomain lambdas could lead to more eye-pleasing results. Some examples would be a weighted average approach where each slice's lambda value is averaged, but more weight is applied to lambda values under a defined threshold.

These are just some of the improvements that could be done to our project in an attempt to improve the computational efficiency, and the accuracy of the animation being plotted between the two images.

8 Bibliography

References

- [1] D. Torres-Muñoz, C. Hernández-Mejía, C. Maldonado-Mendez *et al.*, “Exploring a novel facial animation technique using numerical traced algorithm,” *Multimedia Tools and Applications*, vol. 81, no. 22, pp. 30 961–30 976, sep 2022. [Online]. Available: <https://doi.org/10.1007/s11042-022-12944-7>
- [2] S. Hernandez-Mendez, C. Hernández-Mejía, D. Torres-Muñoz, and C. Maldonado-Mendez, “The optimization of numerical algorithm parameters with a genetic algorithm to animate letters of the sign alphabet,” *Multimodal Technologies and Interaction*, vol. 8, no. 7, 2024. [Online]. Available: <https://www.mdpi.com/2414-4088/8/7/61>